altairsim — A modem terminal on the PMMI MM-103

2026-08-03 · 846eb61

A machine with a **PMMI MM-103 modem** on it, and a small 8080 program that turns it into a **terminal**: what you type on the console goes down the phone line, what the line sends back appears on the screen. Four control keys are stolen to work the modem itself.

```
cd examples/pmmi
altairsim pmmiterm.toml
```

You land at the `altairsim>` prompt. The machine is an 8080, 32K of RAM, an **88-2SIO** for the console (`sio0`, port 10), and the **PMMI MM-103** modem (`pmmi0`) at its factory base `0xC0`. Load the terminal and run it:

```
altairsim> LOAD PMMITERM.HEX
altairsim> RUN 0100
```

It prints a sign-on banner and drops into its terminal loop:

```
PMMI MM-103 TERMINAL
^D DTR  ^S SELF-TEST  ^I STATUS  ^C QUIT
```

The four keys

The loop reads the 2SIO console and the PMMI on every pass. An ordinary key is handed to the PMMI's transmitter; a character the PMMI receives is written to the screen. These four keys never reach the line — they work the modem instead:

Key	Code	What it does
^D	04h	Toggle **DTR** (OUT `0xC3` bit 6) — enable/disable the modem
^S	13h	Toggle the 6860 **Self Test** loopback (OUT `0xC3` bit 4, active low)
^I	09h	Read the **modem-status** byte (IN `0xC2`) and print it in hex
^C	03h	Print `BYE` and `HLT` back to the monitor

`^E` (ATTN) still belongs to the monitor and breaks out at any time; these four belong to the program.

See it echo, with no phone line attached

The modem's `line` starts **disconnected** — `pmmiterm.toml` sets no `dial` / `answer`, so nothing on your host is touched. Type a letter now and **nothing comes back**: the character left the PMMI transmitter and vanished into a dead line. That is the honest behavior of an unconnected modem, and it is the point of the next step.

The 6860 **Self Test** loops the modem's UART onto itself — its transmitter straight back to its own receiver — so it needs no far end. It only engages with the modem enabled, so raise **DTR** first, then Self Test:

```
^D        (DTR on)
^S        (Self Test on — loopback engaged)
```

Now type. **Your keystrokes echo**, because each one leaves the transmitter, the 6860 loops it back, and the terminal writes it to the screen. Press **^S** again to drop the loopback and the echo stops. This is the whole modem data path exercised with nothing but the card.

Press **^I** at any time to see the modem-status byte. With no call up it reads `MS=43` — the idle "clear to send, off-hook, originate" constant the card reports when it has no live phone line to describe.

Finish with **^C**: the program prints `BYE`, halts, and you are back at `altairsim>`.

Placing a real call

To make the terminal talk to something over TCP instead of looping back, give the modem a far end in the machine file — `dial` for the number it originates to, `answer` for a port it picks up on:

```toml
[[board]]
type = "pmmi"
id   = "pmmi0"
port = C0
dial   = "bbs.example.com:23"   # ^D (DTR) + an off-hook write dials this
answer = "2323"                 # ^D arms auto-answer on TCP port 2323
```

With a far end configured, `^D` runs the real handshake: `^I` then shows the live status walking through dialing, ringing, carrier and clear-to-send instead of the idle constant. The board and its

handshake are described in `../../machines/pmmi.toml` and the User Manual.

The source is `PMMITERM.ASM`, assembled into `PMMITERM.HEX` beside it — there is no assembler in the tree, so the listing is there to be read and the HEX is the image the machine loads.